mdexplore

Fast Markdown explorer for Ubuntu/Linux desktop: browse `.md` files in a directory tree and preview fully rendered output instantly.

User Guide

For a use-case/tutorial walkthrough, start with USER-GUIDE.md.

Code Layout

The application still uses `mdexplore.py` as the main entrypoint and primary UI module, but low-risk support code is now split into `mdexplore_app/`:

- `mdexplore.py`: main window, markdown renderer, preview/PDF orchestration, and app entrypoint.
- `mdexplore_app/constants.py`: shared runtime/render constants.
- `mdexplore_app/runtime.py`: runtime environment helpers (config path, default root, GPU/software fallback, print layout knobs).
- `mdexplore_app/search.py`: extracted search tokenization, boolean/NEAR parsing, and per-file hit counting.
- `mdexplore_app/templates.py`: extracted HTML template-asset registry/renderer for preview document shells.
- `mdexplore_app/pdf.py`: PDF footer stamping, blank-page suppression, and PlantUML stderr formatting.
- `mdexplore_app/icons.py`: icon loading/recoloring helpers.
- `mdexplore_app/tree.py`: markdown tree model and delegate (`ColorizedMarkdownModel`, `MarkdownTreeItemDelegate`).
- `mdexplore_app/tabs.py`: custom multi-view tab bar (`ViewTabBar`).
- `mdexplore_app/workers.py`: background worker classes for preview render, PlantUML, and PDF write/stamp steps.
- `mdexplore_app/fast_base64.py`: shared BASE64 helpers with vendor `fastbase64` + `pybase64` acceleration and stdlib fallback.

This is intentionally a first-stage modularization. Most behavior and call flow still lives in `mdexplore.py` so feature risk stays low while the file is gradually decomposed.

Features

- Expandable left-pane directory tree rooted at a chosen folder.
- Shows Markdown files only (`*.md`), while still allowing folder navigation.
- Large right-pane rendered preview with scroll support.
- PlantUML diagram renders run in background workers so markdown preview stays responsive.
- Supports:
  - CommonMark + tables + strikethrough.
  - Markdown rendering defaults to `cmarkgfm` for speed, with automatic fallback to `markdown-it-py` when unavailable/incompatible.
  - TeX/LaTeX math via MathJax.
  - Mermaid diagrams with improved dark-theme contrast.
  - Optional Rust Mermaid backend via `mmdr` (`--mermaid-backend rust`).
  - PlantUML diagrams (asynchronous local render with placeholders).
  - Markdown callouts (`> [!NOTE]` , `> [!TIP]` , `> [!IMPORTANT]` , `> [!WARNING]` , `> [!CAUTION]`).
- Top actions:
  - `Recent` opens a dropdown of up to 35 retained root directories: first 10 shown most-recent-first, then a separator, then up to 25 remaining entries sorted alphabetically. The list refreshes from disk each time the menu is opened so multiple running instances stay in sync. A root is added only after it has been active for at least 30 seconds and then you navigate to another root.
  - `^` moves root up one directory level.
  - `Refresh` rescans the current directory view to pick up new/deleted files.
  - `PDF` exports the current preview to `<filename>.pdf` with centered page numbering (`N of M`). During export, retrievable image sources are inlined as BASE64 data URIs so images render in the PDF instead of relying on external links; broken links are left unchanged.
  - `Add View` creates another tabbed view of the same document at the current top visible line.
  - `Edit` opens the selected file in MarkText (`/usr/bin/marktext`).
- Window title shows the current effective root path.
- Effective-root directory row is always bold:
  - aqua-blue (`#7fdfe8`) when no active search hits are under that scope,
  - yellow with an appended hit-count pill when active search has hits under that scope (`1..99` , then `++`).
- Preview cache keyed by file timestamp and size for fast re-open.
- Inline preview BASE64 images are materialized in parallel per document (deduped by payload hash) to reduce first-load latency on image-heavy files.
- Copy-time BASE64 image conversion warms image targets in parallel before rewrite for faster large-file copy/export workflows.
- Mermaid SVGs are cached in-memory per app run to avoid re-rendering when returning to previously viewed files.
- Mermaid keeps separate in-memory cache modes for GUI (`auto`) and PDF (`pdf`) rendering.

- Navigating back to a cached file still performs a fresh stat check; changed files re-render automatically.
- Startup-generated icon assets are persisted under `~/.cache/mdexplore/icon-cache`:
  - app icon normalization output (from `mdexplor-icon.png`/fallback icon files),
  - two-tone icon recolor outputs used by the tab/tree UI. These are regenerated automatically when source asset identity (path/mtime/size) or render parameters change.
- `F5` refresh shortcut for directory view rescan (same behavior as `Refresh` button).
- `Ctrl++` / `Ctrl+-` / `Ctrl+0` zoom only the preview pane content in/out/reset.
- Preview-only zoom changes briefly show a percentage badge at the top of the preview pane.
- If the currently previewed markdown file changes on disk, preview auto-refreshes and shows a status bar message.
- Status bar reports active long-running work (preview load/render, PlantUML progress, PDF export) and returns to `Ready` instead of staying blank.
- Manual tree/preview pane resizing is preserved across `^` root navigation for the current app run.
- Right-click a Markdown file to assign a highlight color in the tree.
- Highlight colors persist per directory in `.mdexplore-colors.json` files.
- Available file-highlight colors are: Yellow, Green, Blue, Orange, Purple, Light Gray, Medium Gray, and Red.
- View-tab state persists per directory in `.mdexplore-views.json` files for documents that have more than one saved view or a custom tab label.
- Persistent preview text highlights are stored per directory in `.mdexplore-highlighting.json`.
- Files in the tree show a small light-gray tab badge when they currently have more than the default single view.
- Files with persistent preview text highlights show a separate purple marker badge in the tree.
- Markdown tree gutter badges are packed together in a fixed-width strip so the markdown file icon and filename stay aligned even when only some badges apply.
- Right-click tree menu includes `Clear in Directory` (non-recursive) and `Clear All` (recursive) to remove persisted file-highlight colors, and both actions prompt for confirmation before clearing.
- Top-right copy controls are labeled `Copy to: () Clipboard () Directory`, with `Clipboard` selected by default.
- A BASE64 image toggle button sits immediately to the left of `Copy to:`:
  - `off` tooltip: `Turn BASE64 image encoding on`
  - `on` tooltip: `Turn BASE64 image encoding off`
  - toggle state persists in `~/.mdexplore.cfg` across restarts (`copy_base64_images_enabled`).
  - when enabled, copied markdown (clipboard staging or directory copy) converts retrievable image links (local paths and URLs) into embedded BASE64 `data:` URIs.
- In `Clipboard` mode, color buttons copy matching highlighted files and the pin button copies the currently previewed markdown file.
- In `Directory` mode, pin/color actions open a target-folder picker, then copy the file(s) into that folder.

- Directory copy also writes merged metadata entries for copied files into the target folder sidecars: `.mdexplore-colors.json`, `.mdexplore-views.json`, and `.mdexplore-highlighting.json`.
- Search box includes an explicit `X` clear control that clears the query and removes match styling.
- Active search evaluates the markdown files currently visible in the tree (root + expanded branches), and reruns automatically when tree visibility changes (expand/collapse/root/scope navigation).
- Search-hit files in the tree show a yellow hit-count pill in the left gutter.
- Matching filenames are styled bold+italic, and filename-term matches are rendered in yellow text.
- When search is active and a matched file is opened, preview matches are highlighted in yellow and the view scrolls to the first match.
- While dragging the preview scrollbar, mdexplore shows an approximate `current line / total lines` indicator beside the scrollbar handle.
- When search is active, preview scrollbar markers show the vertical positions of highlighted hits; clicking a marker jumps to the nearest hit in that cluster.
- Preview scroll position is remembered per markdown file for the current app session.
- Right-click selected text in the preview pane to use:
  - `Copy Rendered Text` for plain rendered selection text.
  - `Copy Source Markdown` for markdown source content. Copies matching source markdown using direct range mapping first, then selected-text/fuzzy line matching as fallback, and finally the full source file if no match is possible.
- Clipboard copy uses file URI MIME formats compatible with Nemo/Nautilus paste.
- Last effective root plus recent-root history are persisted to `~/.mdexplore.cfg` on root navigation and on exit.
  - Payload format is JSON with keys `default_root`, `recent_roots`, and `copy_base64_images_enabled` (legacy plain-text config is still accepted on read).
  - Recent roots are capped at 35 entries in rolling newest-first storage. Menu presentation is split: 10 most-recent-first, separator, then remaining entries alphabetically.
  - Config writes use a lock file (`~/.mdexplore.cfg.lock`) with non-blocking, momentary locking.
  - If another instance holds the lock during a save attempt, that save is skipped silently.
  - Lock files older than 2 minutes are cleaned up automatically (silently).
  - If no directory is selected at quit time, the most recently selected/expanded directory is used for `default_root`.

Requirements

- Ubuntu/Linux desktop with GUI.
- Python `3.10+`.
- `python3-venv` package available.
- Internet access for Mermaid only when no local Mermaid bundle is available.
- Internet access for MathJax only when no local MathJax bundle is available.
- Java runtime (`java` in `PATH`) for local PlantUML rendering.
- `plantuml.jar` available (vendored path by default, or set `PLANTUML_JAR`).
- Optional: MarkText installed at `/usr/bin/marktext` for `Edit`.
- Optional: `mmdr` in `PATH` (or `MDEXPLORE_MERMAID_RS_BIN`) for Rust Mermaid backend.

Quick Start

From any directory:

```
/path/to/mdexplore/setup-mdexplore.sh
/path/to/mdexplore/mdexplore.sh
```

`setup-mdexplore.sh` is the full bootstrap path. It:

- creates or updates `.venv`
- installs Python dependencies from `requirements.txt`
- verifies tracked UI assets under `assets/ui`
- downloads vendored MathJax, Mermaid, and PlantUML runtime assets if they are missing
- downloads/builds the Rust Mermaid renderer under `vendor/mermaid-rs-renderer`

It is safe to rerun. After bootstrap, use `mdexplore.sh` for normal launches.

When no `PATH` is supplied, the app opens:

1. `default_root` from `~/.mdexplore.cfg` (if valid), otherwise
2. your home directory.

To open a specific root directory:

```
/path/to/mdexplore/mdexplore.sh /path/to/notes
```

What the launcher does:

- Creates `.venv` inside the project if missing.
- Uses `.venv/bin/python` directly (does not alter your current shell session).
- Installs dependencies when `requirements.txt` changes.

- Stores a runtime import-check stamp in `.venv/.runtime-import-check.sha256`.
- Runs runtime import verification when one of these is true:
  - `.venv` is newly created,
  - `requirements.txt` hash changed,
  - runtime import-check stamp is missing or does not match requirements hash.
- If runtime import verification fails, self-heals by reinstalling dependencies.
- Runs the app.

Usage

Wrapper script

```
mdexplore.sh [--mermaid-backend js|rust] [PATH]
```

- `PATH` is optional.
- `--mermaid-backend` is optional. For `mdexplore.sh`, the launcher defaults to `rust` when not specified. (`mdexplore.py` direct runs default to `js`.)
- Supports plain paths and `file://` URIs (for `.desktop` `%u` launches).
- If a file path is passed, mdexplore opens its parent directory.
- If omitted, `default_root` in `~/.mdexplore.cfg` is used (falling back to home directory).
- `--help` prints usage.

Direct Python run

```
python3 -m pip install -r /path/to/mdexplore/requirements.txt
python3 /path/to/mdexplore/mdexplore.py [--mermaid-backend js|rust] [PATH]
```

If `PATH` is omitted for direct run, the same config/home default rule applies.

hfind CLI

`hfind` reuses mdexplore search syntax (AND/OR/NOT, quoted terms, `NEAR(...)`) for file discovery.

Run via wrapper:

```
./hfind.sh [--query QUERY|-q QUERY] [--base|-b] [--content|-c] [--recursive|-r]
[--verbose|-v] [--pdf|-p] [--sort|-s] [--sort-case-sensitive|-S] PATTERN
[PATTERN ...]
```

Run via Python directly:

```
python3 /path/to/mdexplore/hfind.py [--query QUERY|-q QUERY] [--base|-b] [--content|-c] [--recursive|-r] [--verbose|-v] [--pdf|-p] [--sort|-s] [--sort-case-sensitive|-S] PATTERN [PATTERN ...]
```

Notes:

- `--content` / `-c` is optional.
- `--recursive` / `-r` is optional.
- Default filename/query matching target is full discovered path (directories + filename).
- `--base` / `-b` switches query matching target to basename only (filename + extension).
- `--verbose` / `-v` prints matching line(s) under each matched file and highlights hit text in yellow.
- `--pdf` / `-p` enables searching extracted text inside `.pdf` files.
- Default output is progressive: matches print as they are found.
- `--sort` / `-s` waits for full scan, then prints results sorted case-insensitively.
- `--sort-case-sensitive` / `-S` waits for full scan, then prints results sorted case-sensitively.
  - when either sort mode is enabled, `hfind` prints: `One moment please... finding all matches to sort them`
- `NEAR(...)` is strict in both `mdexplore` and `hfind`: a hit is only produced when all NEAR terms occur within the configured 50-word window.
- For content search, inline `...` payloads are ignored in both `mdexplore` and `hfind` to avoid false positives from embedded image data.
- In verbose `NEAR(...)` output, numbering reflects match semantics:
  - lines that independently satisfy `NEAR(...)` are each numbered,
  - lines shown only as contiguous context for a cross-line NEAR window are grouped and only the first line is numbered.
- Quoted terms with exactly one leading and/or trailing literal space use boundary-aware matching at that edge (word boundary, punctuation, or line break can satisfy it).
  - Example: `'Nico '` can match `Nico.`, `Nico)`, `Nico;`, or `Nico` followed by a line break.
  - Example: `' Nico '` can match `\nNico\n` and `His name is Nico.`
  - Exception: this boundary rule applies only to a single leading/trailing space. Terms like `' Nico'` or `'Nico '` require those two literal spaces.
- Short flags are stackable in any order (for example `-cr`, `-rc`).
- If `-q` / `--query` is omitted, the first positional string is treated as the query.
- Base mode checks basename only (including extension, no path).

Examples:

```
./hfind.sh --query "OR(fred, paul)" --content --recursive *.txt
```

Recursively searches from current directory for readable `.txt` files whose path or content contains (case-insensitive) `fred` or `paul`.

```
./hfind.sh -q "OR(fred, paul)" -cr *.txt
./hfind.sh -cr "OR(fred, paul)" *.txt
./hfind.sh --recursive -c "OR(fred, paul)" *.txt
./hfind.sh -crvp "OR(fred, paul)" *.txt
./hfind.sh -crvps "OR(fred, paul)" *.txt
./hfind.sh -crvpS "OR(fred, paul)" *.txt
```

All above are valid shorthand forms.

```
./hfind.sh -rs "conflicted" "./**/*"
./hfind.sh -rS "conflicted" "./**/*"
```

Shows the two sort modes over recursive path matches:

- `-rs` sorts case-insensitively.
- `-rS` sorts case-sensitively.

```
./hfind.sh --query "OR(fred, paul)" *.txt
```

Searches discovered paths only (current directory, non-recursive).

```
./hfind.sh -rb "site" "./**/*"
```

Recursively searches using basename-only matching (legacy filename behavior).

```
./hfind.sh -q "AND('Fred',paul)" /path/to/directory/*.md
```

Searches discovered paths in `/path/to/directory/` (non-recursive) where query requires case-sensitive `Fred` and case-insensitive `paul`.

```
./hfind.sh -rc "NEAR(\"Fred is my friend\", \"is a nice guy\")" "/path/to
my/directory/*.md"
```

Uses escaped double quotes inside a `NEAR(...)` query while recursively searching matching markdown files.

```
./hfind.sh -crv "NEAR(alpha,beta)" "./**/*.md"
```

For each matching file, prints matching line numbers and highlights the triggering terms in yellow. For `NEAR(...)`, this may include multiple nearby lines so both terms are visible.

```
./hfind.sh -cv "'Nico '" "./**/*.md"
./hfind.sh -cv "' Nico '" "./**/*.md"
```

Demonstrates the single-space boundary behavior for quoted terms (including punctuation and line-break boundaries).

Full Bootstrap Script

```
setup-mdexplore.sh [--skip-python] [--skip-assets] [--skip-rust] [--rebuild-rust]
```

- `--skip-python` leaves the existing `.venv` untouched.
- `--skip-assets` skips vendored MathJax / Mermaid JS / PlantUML runtime asset checks.
- `--skip-rust` skips Rust Mermaid bootstrap/build.
- `--rebuild-rust` forces a fresh `cargo build --release --locked` for `mmdr`.

By default the script prefers the vendored `vendor/mermaid-rs-renderer/` checkout if it already exists. If it does not exist, the script clones it from:

```
https://github.com/1jehuang/mermaid-rs-renderer.git
```

If `cargo` is missing, the setup script installs a minimal Rust toolchain through `rustup` so it can build `mmdr` locally.

Headless Regression Tests

Run the GUI regression suite under `xvfb` so `QWebEngineView` has a display:

```
xvfb-run -a .venv/bin/python -m unittest discover -s tests -v
```

The current suites in tests/test_preview_regressions.py, tests/test_search_query_syntax.py, tests/test_pdf_layout_hints.py, tests/test_template_assets.py, tests/test_tab_bar_layout.py, and tests/test_window_layout.py cover:

- saved view-tab round trips on the `test/testdoc.md` fixture,
- left-gutter named-view markers restoring the same saved positions as tabs,
- left-gutter persistent highlight markers navigating to visible highlights,
- right-gutter search markers navigating to visible search hits,

- same-document live preview search coverage for unquoted terms, double/single quotes, trailing-space phrases, implicit AND, function-style AND/OR, `NOT`, and `NEAR(...)` variants,
- search parser/matcher handling for unquoted terms, double-quoted phrases, single-quoted case-sensitive phrases, and literal apostrophes inside double-quoted phrases,
- PDF post-pass TOC detection and landscape/diagram page-flag classification,
- HTML preview-template asset loading and `MarkdownRenderer.render_document()` integration,
- custom tab-bar label budgeting and stale close-button geometry fallback.
- top-bar path-label layout protection so long document paths do not force the main window to stay overly wide.

File Highlights

- In the file tree, right-click any `.md` file.
- Choose a highlight color (`Highlight Yellow`, then `... <Color>`), or clear it.
- Colors are persisted in `.mdexplore-colors.json` in each affected directory.
- Available colors include `Light Gray` and `Medium Gray` in addition to the original color set.
- If a directory is not writable, color persistence fails quietly by design.
- Use top-right `Copy to: () Clipboard () Directory` controls to choose copy destination mode.
- In `Clipboard` mode, color buttons copy files with a given highlight color and the pin button copies the currently previewed markdown file path payload.
- In `Directory` mode, pin/color actions open a folder chooser that defaults to: previously selected destination folder, else current effective root.
- Directory copy writes file content and merges copied-file metadata into target `.mdexplore-colors.json`, `.mdexplore-views.json`, and `.mdexplore-highlighting.json` (creating sidecars when missing, updating existing sidecars by filename when present).
- Right-click a directory (or file row) and use `Clear in Directory` for non-recursive clear in that directory, with confirmation.
- Right-click a directory (or file row) and use `Clear All` for recursive clear under that scope, with confirmation.
- Color-copy match collection is recursive and uses scope in this order: selected directory, else most recently selected/expanded directory, else root.

Preview Text Highlights

- In the preview pane, right-click selected text and choose either `Highlight` or `Highlight Important` to add a persistent text highlight behind that rendered text.
- Normal highlights use a darker subdued purple. Important highlights use a lighter purple so the two categories are visually distinct on the dark theme.
- Selecting part or all of an existing highlighted block and applying the other action converts just that selected range to the chosen highlight type.
- Highlighted preview text also appears as left-side navigation markers in the preview gutter. Important-highlight markers are lighter than normal-highlight markers, and longer markers indicate highlights that span more lines.
- Clicking a highlight marker jumps to the corresponding highlighted block.
- Named views with a saved `Return to beginning` anchor also appear as color-matched left-gutter markers in the preview, layered above highlight markers when the two coincide.
- Right-click inside an existing highlighted block to remove it with `Remove Highlight`.
- If the current selection overlaps existing highlighted text and also includes unhighlighted text, both highlight actions and `Remove Highlight` remain available so the block can be extended, converted, or removed.
- Preview text highlights persist per directory in `.mdexplore-highlighting.json`.
- The tree also mirrors persisted preview highlights with a marker badge so highlighted documents remain easy to spot while browsing directories.
- The tree gutter badge order is: hit-count pill, highlight marker, views badge, markdown file icon.
- Highlight persistence is best-effort: if the directory is not writable, mdexplore fails quietly rather than interrupting preview use.

Preview Zoom

- `Ctrl++` zooms the preview pane in.
- `Ctrl+-` zooms the preview pane out.
- `Ctrl+0` resets preview zoom to `100%`.
- These shortcuts affect only the preview content, not the tree pane, toolbar, or overall window layout.
- Each zoom change briefly shows a percentage badge at the top-center of the preview pane and also reports the same value in the status bar.

PDF Export

- Click `PDF` (between `Refresh` and `Add View`) to export the currently previewed file.
- Output path is the previewed file path with `.pdf` extension in the same directory.
- Export is based on the active document content and print-prepared preview state (markdown + math + Mermaid + PlantUML).
- Export waits briefly for math/diagram/font readiness to reduce rendering artifacts.
- Mermaid PDF behavior is backend-specific:
  - JS backend: Mermaid is rendered through a print-safe monochrome/grayscale path.
  - Rust backend: Mermaid starts from a dedicated PDF SVG set rendered by `mmdr` (default theming), then applies print-safe grayscale normalization (multi-shade, with readable dark text).
- Export auto-scales page content into a print-style layout with top/side margins and an uncluttered footer band.
- Landscape diagram sections are isolated onto dedicated print blocks and use a tighter horizontal margin budget so wide diagrams can make use of the rotated page.
- Headed landscape sections are anchored to the page that actually contains the diagram heading/content, avoiding earlier prose pages being rotated instead.
- Pages that look like a table of contents are never promoted to landscape just because they mention a landscape section heading in the TOC text.
- Footer number font size is matched to the document's dominant scaled body text size.
- Pages are stamped with centered footer numbering as `1 of N`, `2 of N`, etc.

PDF Layout Tuning

PDF pagination and diagram sizing are controlled by a small group of constants near the top of `mdexplore.py`. These are the first settings to adjust if PDF output starts making poor keep/spill/landscape choices.

Most important knobs:

- `MIN_PRINT_DIAGRAM_FONT_PT`
  - Lower bound for the largest font in a diagram when mdexplore tries to keep that diagram on a single page.
  - Lower it if you want tall diagrams to stay on one page more often.
  - Raise it if you prefer multi-page spill over small text.
- `MAX_PRINT_DIAGRAM_FONT_PT`
  - Upper bound for diagram enlargement in PDF output.
  - Lower it if diagrams are printing too large.
  - Raise it cautiously if diagrams look too small even when there is space.
- `PDF_PRINT_WIDE_DIAGRAM_ASPECT_RATIO`
  - Aspect-ratio threshold for promoting a diagram to landscape.
  - Lower it if wide diagrams are not rotating when they should.

    - Raise it if too many diagrams are switching to landscape.
- `PDF_PRINT_WIDE_DIAGRAM_LANDSCAPE_GAIN`
    - Minimum width gain required before landscape is preferred.
    - Lower it if landscape should be chosen more aggressively.
    - Raise it if portrait should remain the default more often.
- `PDF_PRINT_PLANTUML_LANDSCAPE_ASPECT_RATIO`
    - Separate PlantUML-specific landscape trigger.
    - Useful because PlantUML often benefits from landscape earlier than Mermaid.
- `PDF_PRINT_HORIZONTAL_MARGIN_PX`
- `PDF_PRINT_VERTICAL_MARGIN_PX`
    - Effective printable margin budget used by the hidden print-layout solver.
    - Lower them to let diagrams use more page area.
    - Raise them if output feels cramped against the page edges.
- `PDF_PRINT_HEADING_TO_DIAGRAM_GAP_PX`
    - Vertical spacing reserved between a heading cluster and the diagram it governs.
    - Raise it if headed sections feel visually crowded.
    - Lower it if headed diagrams are spilling to the next page too eagerly.
- `PDF_PRINT_LAYOUT_SAFETY_PX`
    - Extra safety padding used to avoid borderline fits that clip in Chromium.
    - Lower it if diagrams are being pushed into spill mode too early.
    - Raise it if “just barely fits” cases are clipping or producing unstable output.

Heuristic guidance:

- If a diagram is being split across pages but you would accept smaller text, lower `MIN_PRINT_DIAGRAM_FONT_PT`.
- If a diagram is tiny but should clearly be landscape, first adjust `PDF_PRINT_WIDE_DIAGRAM_ASPECT_RATIO` or `PDF_PRINT_WIDE_DIAGRAM_LANDSCAPE_GAIN`.
- If headings are being orphaned above diagrams, leave the font knobs alone and look instead at the page-geometry and spacing constants.
- If many diagrams are near the right answer but consistently a little too big or too small, adjust margins before changing font caps.

Recommended workflow when tuning:

1. Change one constant at a time.
2. Regenerate the specific problematic PDF.
3. Recheck both the target document and one known-good document.
4. Prefer small adjustments; many of these settings interact.

The hidden PDF preflight logic consumes these constants through a Python-to-JS configuration handoff, so changing the constants in `mdexplore.py` is the intended way to tune print behavior.

Known TODOs

- Diagram interaction state restore is not yet reliable across document switches:
  - Mermaid zoom/pan state may not restore consistently when returning to a file.
  - PlantUML zoom/pan state may not restore consistently when returning to a file.
- This is tracked as an explicit TODO for future maintenance; core markdown rendering, search, highlighting, and PDF export remain functional.

Multiple Views

- Click `Add View` to create another tab for the same currently previewed document.
- New tabs inherit the current view's top visible line/scroll position.
- By default, tab labels show the current top-most visible source line number for that tab.
- Tabs show a small left-side position bargraph indicating where that view sits within the document.
- Tabs use a fixed soft-pastel color sequence based on the order each view was opened.
- Tabs can be dragged to reorder without changing each tab's assigned color.
- Right-click a tab to assign a custom label (including spaces) up to 48 characters.
- If a longer label is entered, mdexplore truncates it to the first 48 characters.
- Entering a blank custom label restores the default dynamic line-number label for that tab.
- When a tab receives a custom label, mdexplore stores that tab's current scroll position and top visible source line as the tab's saved beginning.
- Right-click a custom-labeled tab to use `Return to beginning`, which jumps that tab back to the stored label-time location.
- Custom-labeled tabs also show a refresh icon beside the close button; clicking it resets that tab's saved beginning to the current scroll position/top line, the same way relabeling the tab does.
- Relabeling a custom-labeled tab with different text resets the saved beginning to the scroll position at the time of relabeling.
- When a new view is added and the palette wraps, mdexplore skips any color already used by open tabs.
- If you switch to another markdown file and later return in the same app run, that file's tabs restore with their prior order and selected tab.
- View-tab state also persists across app restarts in `.mdexplore-views.json` beside the document directory, keyed by markdown filename.
- For custom-labeled tabs, `.mdexplore-views.json` also persists the stored label-time beginning location used by `Return to beginning`.
- If custom labels make the tab strip too wide for the window, tab scrolling is enabled through the tab bar's built-in scroll buttons.
- Only documents with more than one saved view, or with a custom tab label, are written to `.mdexplore-views.json`; untouched single-view documents continue to use the default one-tab state.
- The tab strip is hidden when there is only one unlabeled default view.
- If only one view remains and it has a custom label, its tab stays visible so the custom label and `Return to beginning` action remain available.
- Closing that sole remaining custom-labeled tab clears the custom label and bookmark, then returns the document to the hidden default single-view state.
- Tabs are closeable with `X`; at least one tab is always kept open.
- Maximum views per document: `8`.
- Named-view left-gutter markers use the same restore path as selecting the corresponding tab, so marker navigation lands at the same saved location.

Markdown Callouts

- mdexplore supports GitHub/Obsidian-style callouts written as blockquotes.
- Supported types: `NOTE`, `TIP`, `IMPORTANT`, `WARNING`, `CAUTION` (case-insensitive).
- `INFO` is accepted and styled as a `NOTE` callout.
- Custom titles are supported: `> [!WARNING] Custom title`.
- `+` / `-` markers are accepted in syntax, but callouts are rendered as non-collapsible boxes.

Example:

```
> [!NOTE]
> This is a note callout with **markdown** content.
```

Search and Match Highlighting

- Use `Search and highlight:` to match markdown files currently visible in the tree (root + expanded directories).
- While search is active, expand/collapse and directory/root scope changes automatically rerun the search against the newly visible set.
- Matching files are shown bold+italic in the tree.
- If filename terms match, filename text is rendered in yellow.
- The effective-root directory label is bold aqua when idle, and becomes yellow with an appended hit-count pill when active search has hits under that scope.
- Press `Enter` in the search field to run search immediately (skip debounce).
- Clicking the `X` in the search field clears search text and removes match styling.
- Opening a matched file while search is active highlights matching text in yellow in the preview and scrolls to the first highlighted match.
- Preview scrollbar markers show where highlighted hits occur within the document; clicking a marker jumps to the nearest hit in that marker cluster.
- For `NEAR(...)` queries, preview highlighting is constrained to qualifying NEAR windows rather than every standalone occurrence of those terms elsewhere in the document.
- Non-quoted terms are case-insensitive.
- Double-quoted terms are case-insensitive and preserve spaces.
- Single-quoted terms are case-sensitive and preserve spaces.
- Only the quote character that opens a phrase closes it; the other quote character remains literal inside the phrase.
- Function-style operators accept both no-space and spaced forms before `(`: `NEAR(...)` / `NEAR (...)`, `OR(...)` / `OR (...)`, `AND(...)` / `AND (...)`, and `NOT(...)` / `NOT (...)`.
- Legacy `CLOSE(...)` remains accepted for backward compatibility, but is normalized internally to canonical `NEAR(...)`.
- `AND(...)`, `OR(...)`, and `NEAR(...)` accept comma-delimited, space-delimited, or mixed argument lists.
- `AND(...)` and `OR(...)` are variadic and can take 2+ arguments.
- `NEAR(...)` requires 2+ terms and matches only when all terms appear within 50 words of each other in file content.
- `NEAR(...)` requires distinct qualifying occurrences for each listed term; one text start cannot satisfy two different required terms.
- For single-word `NEAR(...)` terms, proximity matching uses word boundaries, so `the` does not qualify via the `The` in `They`.
- For `NEAR(...)` queries, the tree hit-count pill counts qualifying NEAR windows in the file, not the number of individual highlighted term spans.

Search examples:

```
joe
```

Matches `joe`, `JOE`, `JoE`, etc. (filename or content).

```
"Anne Smith"
```

Case-insensitive exact phrase match.

```
'Anne Smith'
```

Case-sensitive exact phrase match.

```
"Program Director's RAG"
```

Case-insensitive exact phrase match, with the apostrophe treated as literal text.

```
OR (Joe, "Fred")
```

Matches files containing either `Joe` (any case) or phrase `"Fred"` in any case.

```
NEAR(Fred "Anne Smith" Joe)
```

Matches only if all listed terms occur within 50 words of each other.

```
NEAR('The ', the)
```

Requires a distinct later/earlier standalone `the`; the `The` in `They` does not qualify.

```
Joe "Anne Smith" NOT draft
```

Implicit `AND`: equivalent to `Joe AND "Anne Smith" AND NOT draft`.

```
AND(alpha beta gamma)
```

Requires all three terms to match (equivalent to `alpha AND beta AND gamma`).

PlantUML Local Configuration

The app renders PlantUML blocks locally with `plantuml.jar`.

- Default jar path: `/path/to/mdexplore/vendor/plantuml/plantuml.jar`

- Override jar path with `PLANTUML_JAR`:

```
PLANTUML_JAR=/path/to/plantuml.jar /path/to/mdexplore/mdexplore.sh
```

Behavior details:

- Markdown preview loads immediately with `PlantUML rendering...` placeholders.
- Each diagram is replaced automatically as soon as it completes.
- Diagram replacements are applied in-place so scroll position is preserved.
- PlantUML SVG `data:` URIs are BASE64-encoded through guarded helpers; if vendored `fastbase64` encode output is malformed, mdexplore automatically falls back to `pybase64`/stdlib encode so diagrams still render.
- Failed diagrams show `PlantUML render failed with error ...` plus detailed stderr context (including line number when PlantUML provides one).
- Diagram progress continues while you browse other files; returning shows completed progress.

MathJax Local-First Configuration

Math rendering is local-first:

- mdexplore first tries a local `tex-svg.js` bundle (best rendering quality).
- If no local SVG bundle is found, it falls back to local `tex-mml-chtml.js`.
- If no local bundle is found, it falls back to CDN.

Local lookup order:

1. `MDEXPLORE_MATHJAX_JS` (explicit file path)
2. `mathjax/es5/tex-svg.js` under the repo
3. `mathjax/tex-svg.js` under the repo
4. `assets/mathjax/es5/tex-svg.js` under the repo
5. `vendor/mathjax/es5/tex-svg.js` under the repo
6. System paths such as `/usr/share/javascript/mathjax/es5/tex-svg.js`
7. Local CHTML bundle paths (`tex-mml-chtml.js`) as fallback

Example override:

```
MDEXPLORE_MATHJAX_JS=/absolute/path/to/tex-svg.js
/path/to/mdexplore/mdexplore.sh
```

Mermaid Local-First Configuration

Mermaid rendering is local-first:

- mdexplore first tries a local `mermaid.min.js` bundle.
- If no local bundle is found, it falls back to CDN.
- If `--mermaid-backend rust` is used, mdexplore uses `mmdr` instead of Mermaid JS for diagram SVG generation.

Local lookup order:

1. `MDEXPLORE_MERMAID_JS` (explicit file path)
2. `mermaid/mermaid.min.js` under the repo
3. `mermaid/dist/mermaid.min.js` under the repo
4. `assets/mermaid/mermaid.min.js` under the repo
5. `assets/mermaid/dist/mermaid.min.js` under the repo
6. `vendor/mermaid/mermaid.min.js` under the repo
7. `vendor/mermaid/dist/mermaid.min.js` under the repo
8. System paths such as `/usr/share/javascript/mermaid/mermaid.min.js`

Example override:

```
MDEXPLORE_MERMAID_JS=/absolute/path/to/mermaid.min.js
/path/to/mdexplore/mdexplore.sh
```

Rust backend executable override:

```
MDEXPLORE_MERMAID_RS_BIN=/absolute/path/to/mmdr /path/to/mdexplore/mdexplore.sh
--mermaid-backend rust
```

Markdown Engine and BASE64 Performance Knobs

- Markdown engine selection:
  - `MDEXPLORE_MARKDOWN_ENGINE=cmark` (default): uses `cmarkgfm`.
  - `MDEXPLORE_MARKDOWN_ENGINE=markdown-it`: forces `markdown-it-py`.
  - If `cmarkgfm` is unavailable at runtime, mdexplore automatically falls back to `markdown-it-py`.

Example:

```
MDEXPLORE_MARKDOWN_ENGINE=markdown-it /path/to/mdexplore/mdexplore.sh
```

- BASE64 image worker threads (preview data-URI materialization + copy-time prefetch):
  - `MDEXPLORE_BASE64_IMAGE_THREADS=<N>`
  - Default is CPU-based (`max(2, min(24, cpu_count * 2))`).

Example:

```
MDEXPLORE_BASE64_IMAGE_THREADS=24 /path/to/mdexplore/mdexplore.sh
```

- BASE64 codec path:
  - Decode path prefers `pybase64`, with adaptive routing to vendored `vendor/fastbase64` for payload-size sweet spots.
  - Encode path may use vendored `fastbase64`, but output is validated (length/padding/alphabet). Invalid vendor output is discarded and mdexplore falls back to `pybase64`/stdlib automatically.
  - Adaptive benchmark state persists in `fastbase64-adaptive.json`.
  - Optional vendor-vs-fallback benchmark telemetry is written to `fastbase64-benchmark.log` when the debug dual-path switch is enabled in `mdexplore_app/fast_base64.py`.
  - If accelerated backends are unavailable, helpers fall back to Python stdlib `base64`.

Debug Logging

- `mdexplore.py --debug` enables verbose debug logging to `mdexplore.log` in the project directory.
- `mdexplore.sh` exposes the same behavior through the `DEBUG_MODE` variable near the top of the script. Set `DEBUG_MODE=true` to pass `--debug` into the app.
- When debug logging is disabled, `mdexplore.log` is not written.
- The application debug log trims itself to the most recent `10,000` lines.
- The non-interactive launcher log remains separate at `~/.cache/mdexplore/launcher.log` and trims to `1,000` lines.

Render and Rules Maps

- See DEVELOPERS-AGENTS.md for the consolidated deep maps:
  - formal behavior/rule hierarchy,
  - render/caching and PDF/export control flow,
  - decision tables and invariants,
  - agent-facing maintenance workflow.
- See UML.md for subsystem-level architecture, class relationships, and activity diagrams.

Project Structure

```
mdexplore.py                  # Qt application, file tree, renderer integration
mdexplore.sh                  # launcher (venv create/install/run)
setup-mdexplore.sh            # full bootstrap script (venv/assets/mmdr)
mdexplore.desktop.sample      # sample desktop launcher entry for user
customization
mdexplore_app/                # extracted support modules for search, tree,
tabs, icons, PDF, runtime, template/JS assets, and workers
tests/                        # headless/unit regressions for preview, search,
template assets, and tab layout
assets/js/                    # externalized preview/PDF JavaScript templates
loaded into a startup registry at startup
assets/js/pdf/                # PDF export/preflight/restore JavaScript
templates
assets/js/preview/            # preview search/highlight/selection/context-menu
JavaScript templates
assets/templates/             # externalized HTML preview document templates
loaded into a startup registry
assets/templates/preview/     # preview HTML shell/template assets
assets/ui/                    # local UI icons/font assets used by tree, tabs,
copy controls, and search pills
mdexplor-icon.png             # primary app icon asset kept at repo root for
desktop launchers
requirements.txt              # Python runtime dependencies
README.md                     # project docs
DEVELOPERS-AGENTS.md          # developer and coding-agent maintenance guide
UML.md                        # subsystem-level architecture/class/activity
diagrams
LICENSE                       # MIT license
```

Development

Basic local checks:

```
bash -n mdexplore.sh
python3 -m py_compile mdexplore.py
```

Troubleshooting

If you see `ModuleNotFoundError: No module named 'PySide6.QtWebEngineWidgets'`:

- Re-run `mdexplore.sh`; if runtime import verification is triggered and fails, it auto-reinstalls dependencies.
- If verification was previously stamped but the venv later drifted, force a recheck by removing:
  - `/path/to/mdexplore/.venv/.runtime-import-check.sha256` then run:
  - `/path/to/mdexplore/mdexplore.sh`
- If it still fails, run:
  - `rm -rf /path/to/mdexplore/.venv`
  - `/path/to/mdexplore/mdexplore.sh`

If `Edit` does nothing:

- Ensure MarkText is installed.
- Verify `/usr/bin/marktext` exists and is executable.

If the dock/menu still shows an old app icon:

- Ensure your launcher contains:
  - `Icon=/absolute/path/to/mdexplor-icon.png`
  - `StartupWMClass=mdexplore`
- Refresh desktop entries:
  - `update-desktop-database ~/.local/share/applications`
- Remove old pinned `mdexplore` favorites from the dock and pin it again.
- Log out/in if the shell still shows the previous cached icon.

If running the launcher appears to do nothing:

- Watch terminal output; the launcher now prints setup/launch status.
- First dependency install can take time because Qt packages are large.
- Ensure you are in a GUI session (`DISPLAY` or `WAYLAND_DISPLAY` must be set).
- For desktop/dock launches (`Terminal=false`), check launcher log:
  - `~/.cache/mdexplore/launcher.log`
  - log is auto-trimmed to the most recent 1000 lines

Security Notes

- Markdown HTML is enabled (`html=True`), so preview untrusted Markdown with care.
- Mermaid uses local-first loading with CDN fallback.
- MathJax uses local-first loading with CDN fallback.

Contributing

1. Keep changes focused and small.
2. Run syntax checks before submitting.
3. Update docs (`README.md` , `DEVELOPERS-AGENTS.md` , and `UML.md` when relevant) when behavior changes.

License

This project is licensed under the MIT License. See `LICENSE` .